Recruitment Management System

Database Design & ER Diagram — Interview Preparation Guide

41 Entities · 65 Relationships · PostgreSQL 14+
Dual-Mode Platform: Full-Time Hiring + Freelance Marketplace

A complete walkthrough of every table, every relationship,
every design decision — and how to defend each one.

Contents

1. The System in One Page
1.1 What the platform does · 1.2 The two flows · 1.3 The four actors

2. Design Principles Behind the Schema
2.1 Normalization · 2.2 Where I denormalized and why · 2.3 Naming conventions · 2.4 Key strategy

3. Entity Reference — All 41 Tables
3.1 Identity & Profiles (8) · 3.2 Skills (4) · 3.3 Jobs & AI Matching (4)
3.4 Full-Time Pipeline (5) · 3.5 Freelance Engagement (5) · 3.6 Money (6)
3.7 Communication (4) · 3.8 Trust & Admin (5)

4. Relationships & Cardinality Explained

5. Eight Design Decisions You Must Be Able to Defend

6. Constraints — What the Database Enforces

7. Indexing & Query Performance

8. End-to-End Walkthroughs with SQL

9. Interview Question Bank (40 Questions)

10. Known Trade-offs — Answering "What Would You Change?"

11. Glossary of Terms

1. The System in One Page

1.1 What the platform does

The Recruitment Management System is a two-sided marketplace that supports **two fundamentally different hiring models within a single schema**. That dual nature is the single most important thing to communicate in an interview, because almost every interesting design decision in the database follows from it.

In the **full-time flow**, a recruiter posts a job description, candidates apply, and the recruiter moves each candidate through a configurable pipeline of stages — screening, interview, and so on — before extending an offer letter. The relationship ends when the candidate is hired; the platform earns money from listing and profile fees.

In the **freelance flow**, a client posts a project with a budget, freelancers submit competing proposals, and the client accepts one. That acceptance creates a binding contract broken into milestones. Money is placed in escrow up front, work is delivered through chat, and funds are released milestone by milestone. The platform earns a commission on each release, and an admin arbitrates disputes.

> **The framing sentence to open with:** "It's a recruitment platform with two parallel flows — permanent hiring with a stage-based pipeline, and freelance projects with milestone-based escrow payments. They share users, jobs, skills and the AI matching engine, but diverge completely after the application stage. The schema is designed so that shared concepts stay in one table and the divergent parts get their own."

1.2 The two flows side by side

Concept	Full-Time Flow	Freelance Flow
Job posting	`jobs` with `job_type = 'FULL_TIME'`, salary band	`jobs` with `job_type = 'FREELANCE'`, budget + fixed/hourly
Candidate expresses interest	`applications`	`proposals` (includes a bid amount)
Recruiter reaches out first	`invitations`	`invitations` (same table, shared)
Progress tracking	`hiring_stages` + `stage_movements`	`milestones` within a `contract`
Meetings	`interviews`	Chat only
Agreement artefact	`offers` (offer letter)	`contracts`
Money movement	`payments` — platform fees only	`escrows` → `wallets` → `payouts`
Conflict handling	None needed	`disputes`, resolved by admin
Terminates when	Offer accepted / rejected	Final milestone paid + reviews exchanged

1.3 The four actors

Actor	What they do	Tables they own
Candidate	Builds a profile and resume, applies to full-time jobs, bids on freelance projects, delivers milestone work, gets paid	`candidate_profiles`, `resumes`, `freelance_portfolios`, `applications`, `proposals`
Recruiter / Client	Posts jobs, configures pipeline stages and AI weights, reviews AI-ranked candidates, sends invites and offers, funds escrow, approves work	`jobs`, `hiring_stages`, `ai_weightages`, `invitations`, `offers`
Admin	Resolves disputes, bans or suspends users, views platform revenue, configures global settings	`disputes`, `moderation_actions`, `platform_settings`, `audit_logs`
System	Computes AI match scores, writes ledger entries, emits notifications, records audit trail	`match_scores`, `transactions`, `notifications`

> **A trap to avoid.** Do not say "the recruiter and the client are different users." They are the same role — `RECRUITER` — wearing different hats depending on `job_type`. Modelling them as separate entities would duplicate company data and force two login paths. If an interviewer pushes on this, the answer is: the behaviour differs, but the identity and the data do not, so one table with a discriminator is correct.

2. Design Principles Behind the Schema

2.1 Normalization — and how far I took it

The schema is in **Third Normal Form** throughout, with a handful of deliberate, documented exceptions. Being able to explain normal forms using your own tables — rather than the textbook student/course example — is one of the fastest ways to sound credible.

First Normal Form (1NF): no repeating groups, every column atomic

The original draft stored `skills` as a single comma-separated string on the candidate profile. That violates 1NF and breaks the product: you cannot index it, you cannot reliably search "candidates who know PostgreSQL" without matching "PostgreSQL Administration" by accident, and you cannot attach proficiency to an individual skill. The fix was to extract `skills` as a reference table and join through `candidate_skills`.

Second Normal Form (2NF): no partial dependency on a composite key

`candidate_skills` has the logical composite key `(candidate_id, skill_id)`. Its attributes — `proficiency`, `years_of_experience` — depend on *both* halves, which is exactly what 2NF requires. The skill's own name depends only on `skill_id`, so it lives in `skills`, not here. That is 2NF in a sentence, using your own schema.

Third Normal Form (3NF): no transitive dependency

An early version had `company_name` and `website` directly on `recruiter_profiles`. But those depend on the company, not on the recruiter — a transitive dependency. With three recruiters from the same firm you get three copies of the website, and an update anomaly the moment one changes. Extracting `companies` resolves it and unlocks a real feature: a company page listing all its open roles.

2.2 Where I broke normalization on purpose

Interviewers respect deliberate denormalization far more than accidental purity. Four columns in this schema are denormalized, and each has a distinct justification.

Column	Why it is duplicated
`freelance_portfolios.avg_rating` `.total_contracts`	**Read performance.** Search results show a rating on every card. Computing `AVG(rating)` across the reviews table for fifty freelancers on every search page is wasteful. Recomputed by trigger or job whenever a review is published.
`applications.snapshot_ai_score`	**Legal defensibility.** If a rejected candidate challenges the decision, you must show the score *as it stood at the moment of rejection* — not as recomputed after the model was retrained. This is a point-in-time record, not a cache.
`transactions.balance_after`	**Auditability.** A financial ledger should let an auditor verify each line without replaying the whole history. Standard practice in accounting systems.
`wallets.available_balance`	**Concurrency control.** Deriving the balance from `SUM(transactions)` on every read is correct but slow, and gives you no single row to lock. Keeping a materialized balance provides a lock target for `SELECT ... FOR UPDATE`, with periodic reconciliation against the ledger.

> **How to phrase this:** "I normalized to 3NF as the default, then denormalized in four specific places. Each one has a written reason — two for read performance, one for legal point-in-time accuracy, one for concurrency control — and each has a defined path back to the source of truth if it drifts."

2.3 Naming conventions

Rule	Example	Reasoning
Tables plural, snake_case	`candidate_profiles`	A table holds many rows; snake_case avoids quoting in Postgres, which folds unquoted identifiers to lowercase
Primary key always `id`	`jobs.id`	Predictable for ORMs and generic tooling
Foreign key = singular table + `_id`	`job_id`	The join target is readable without opening the schema
Booleans prefixed `is_` / `has_`	`is_verified`	Reads as a predicate at the call site
Timestamps suffixed `_at`	`created_at`	Distinguishes instants from dates and durations
Junction table = both sides	`candidate_skills`	Its purpose is obvious from the name alone
Money as `DECIMAL`, never `FLOAT`	`amount DECIMAL(12,2)`	Binary floating point cannot represent 0.10 exactly; errors compound across a ledger

2.4 Why UUID primary keys instead of auto-increment integers

This comes up often, and there is a real trade-off to acknowledge rather than a single correct answer.

Reason to use UUID	Cost you accept
IDs are not guessable — a sequential integer in a URL invites enumeration of every job or application	16 bytes per key versus 4 or 8, which inflates every index
IDs can be generated client-side or by a service before the row is inserted	Random UUIDv4 values scatter B-tree inserts, hurting write locality
Merging data across environments or shards never collides	Harder to read and quote in a support conversation
Leaks no business information — a sequential ID tells a competitor how many jobs you host	Slightly slower joins at very large scale

The honest closing line: "*For a marketplace where IDs appear in URLs and are shared between parties, non-enumerable keys were worth the storage. If write throughput became the bottleneck I would move to UUIDv7, which is time-ordered and restores insert locality while keeping non-guessability.*" Mentioning UUIDv7 signals that you have thought past the textbook answer.

3. Entity Reference — All 41 Tables

Each entry below gives the table's **purpose**, its **defining columns**, the **design reasoning**, and where relevant the **question an interviewer is likely to ask about it**. Tables marked OPTIONAL are extensions beyond the minimum viable schema — mentioning that you scoped them deliberately shows product judgement.

3.1 Identity & Profiles

users

The single authentication record for every human on the platform, regardless of role. Everything that requires a login points here.

Column	Notes
`email` UNIQUE	The login identifier. Store lowercased and trimmed, or the same person registers twice with different capitalisation.
`password_hash`	Never `password`. bcrypt or Argon2id, never MD5 or SHA-256 alone — those are fast, which is precisely wrong for passwords.
`role`	`ADMIN | RECRUITER | CANDIDATE`. Drives authorization and which profile row exists.
`account_status`	`ACTIVE | SUSPENDED | BANNED | DELETED`. Powers admin moderation without deleting rows.
`deleted_at`	Soft delete. A hard `DELETE` would cascade away contracts and payment records you are legally required to retain.

Design note. Authentication data is separated from profile data because the three roles need wildly different attributes. Putting `company_name` and `resume_url` on the same table would leave most columns `NULL` for most rows — a classic sparse-table smell.

Likely question: "*Why not one table per role?*" Because login, password reset, email verification, notifications and moderation are identical across roles. Splitting them would triple that logic and make a global unique email constraint impossible to enforce.

candidate_profiles

Everything about a job-seeker that is not authentication: headline, location, education, experience, and their availability preferences.

Column	Notes
`user_id` UNIQUE FK	Unique enforces the one-to-one relationship at the database level, not just in application code.
`open_to_full_time` `open_to_freelance`	Two independent booleans, not one enum — a candidate can genuinely want both, and forcing a choice would be a product bug encoded in the schema.
`full_time_fee_paid`	Reflects the mind-map requirement that candidates pay a platform fee to access full-time roles.
`primary_resume_id` FK	Points at the current version in `resumes`. Replaced the earlier plain `resume_url` text column.

Design note. The relationship to `users` is `1 → 0..1`, not `1 → 1`. A recruiter has no candidate profile, so the row must be genuinely optional. Getting this multiplicity wrong is one of the most common errors in student ER diagrams.

freelance_portfolios

The freelance-specific extension of a candidate profile: rate, bio, availability, and reputation.

Column	Notes
`candidate_id` UNIQUE FK	Extends the candidate rather than the user, because only candidates freelance.
`hourly_rate`, `currency`	Currency stored alongside every monetary amount — never assume one currency.
`avg_rating`, `total_contracts`	Denormalized for search performance. See §2.2.
`is_available`	Lets a freelancer stop appearing in invite lists without deleting their profile.

Likely question: "*Why is this separate from candidate_profiles rather than nullable columns on it?*" Because roughly half of candidates never freelance, so those columns would be `NULL` for half the table. Separating them keeps the main profile row narrow — which matters, since it is read on every page — and makes "is this person a freelancer?" a row-existence check rather than a null check across five columns.

recruiter_profiles

The hiring-side profile: which company the person represents and in what capacity.

Column	Notes
`user_id` UNIQUE FK	One recruiter profile per user.
`company_id` FK	Nullable so an independent recruiter can operate without a company record.
`designation`	Job title within the company, shown for trust signalling.

companies

The employing organisation. Extracted from `recruiter_profiles` to eliminate the transitive dependency described in §2.1.

Why it earns its own table: multiple recruiters share one employer; verification is a company-level fact, not a per-recruiter one; and it enables a company profile page aggregating all open roles — a real feature that would be impossible with a plain text column.

admin_profiles

Administrative capabilities, held as a `jsonb` permission set.

Design note. The original class diagram made `Admin` a *subclass* of `User` while candidates and recruiters were composition. That inconsistency is worth calling out yourself before an interviewer finds it: mixing inheritance and composition for parallel concepts means two different code paths for the same idea. All three are now composition.

Likely question: "*Why JSON for permissions instead of a roles table?*" Honest answer: for a small fixed admin team the flexibility outweighs the structure, and it avoids three extra tables. If the admin team grew or permissions needed to be queryable — "which admins can issue refunds?" — I would migrate to a proper `roles` / `permissions` / `role_permissions` RBAC model.

resumes OPTIONAL

Version history of a candidate's uploaded resumes.

Column	Notes
`version`	`UNIQUE(candidate_id, version)` — sequential per candidate.
`is_primary`	Partial unique index guarantees exactly one primary per candidate.
`parsed_text`	Extracted plain text. This is what the AI matcher actually reads — parsing a PDF on every score computation would be far too slow.

Why versioning matters: `applications.resume_id` pins the exact version that was submitted. Without it, a candidate who updates their resume retroactively changes what every past recruiter believes they reviewed — which is both confusing and, in a hiring-decision context, potentially a compliance problem.

portfolio_projects & portfolio_project_skills OPTIONAL

Individual work samples with their own tech stacks, replacing a single `portfolio_url` text field.

Why it matters: the tech stack per project reuses the shared `skills` table, which means "find a freelancer who has actually shipped something in React" becomes a joinable query rather than a full-text guess. This is the 1NF argument applied a second time, and reusing `skills` rather than inventing a parallel tag table is the detail worth pointing out.

3.2 Skills

skills

The controlled vocabulary — one canonical row per skill, shared by candidates, jobs and portfolio projects.

Design note. `is_approved` supports a moderation workflow: users may propose new skills, but unapproved entries stay out of search facets until an admin confirms them. Without this you accumulate "React", "ReactJS", "React.js" and "react" as four distinct skills, and every filter silently under-reports.

candidate_skills & job_skills

Junction tables resolving the many-to-many relationships between skills and the two things that reference them.

Aspect	candidate_skills	job_skills
Resolves	Candidate ↔ Skill	Job ↔ Skill
Extra attributes	`proficiency` (1–5), `years_of_experience`	`is_mandatory`, `weight`
Unique key	`(candidate_id, skill_id)`	`(job_id, skill_id)`

The key teaching point. These are not "just join tables". Each carries attributes that belong to the *relationship itself* rather than to either side — proficiency is not a property of the candidate or of the skill, but of the pairing. This is the textbook definition of an *associative entity*, and naming it as such is a strong signal in an interview. They also feed the matching algorithm directly: `is_mandatory` lets you hard-filter, and `weight` lets you rank.

3.3 Jobs & AI Matching

jobs

A single table serving both full-time postings and freelance projects, discriminated by `job_type`.

Column	Notes
`job_type`	`FULL_TIME | FREELANCE` — the discriminator that determines which other columns must be populated.
`salary_min`, `salary_max`	Full-time only. A range, not a single figure, because that is how roles are actually advertised.
`budget`, `budget_type`	Freelance only. `FIXED | HOURLY` changes how proposals are interpreted.
`status`	Includes `PENDING_PAYMENT` — a job stays invisible until the recruiter's posting fee clears.

Design note. A `CHECK` constraint enforces that full-time rows carry salary fields and no budget, and freelance rows the reverse. Without it nothing stops a row that is simultaneously both, and every downstream query has to defend against nonsense.

Likely question: "*Why one jobs table instead of full_time_jobs and freelance_jobs?*" This is a genuine judgement call — say so. They share about eighty percent of their columns, and far more importantly they share their relationships: invitations, AI weightages, match scores and skill requirements all point at "a job". Splitting the table would double each of those relationships, or force a polymorphic reference that the database cannot enforce with a foreign key. The cost of the single table is that some columns are null for some rows, which a CHECK constraint contains. That trade — a few nullable columns against doubling six relationships — is worth taking.

ai_weightages

Per-job configuration of how the AI match score is computed: how much skills matter versus experience, education and projects.

Why per-job rather than global: a senior architect role should weight experience heavily; a graduate scheme should weight education. Making this configurable per posting is exactly the "Add the AI score weightage" node in the mind map, and it is a genuinely good product decision worth highlighting.

The constraint that makes it correct: `CHECK (skill + experience + education + project = 100)`. Without it a recruiter can enter weights summing to 40 and every score silently comes out low, with no error anywhere — the worst class of bug, because nothing appears broken.

match_scores

The computed compatibility between one candidate and one job.

Column	Notes
`score`	0–100, enforced by CHECK.
`breakdown jsonb`	Per-factor sub-scores. Essential for explainability — a recruiter asking "why is this candidate 72?" deserves an answer, and increasingly regulators do too.
`model_version`	Which scoring model produced this. Lets you compare model revisions and roll back.
`is_current`	Partial unique index guarantees exactly one live score per candidate–job pair while retaining history.

> **This is the single most important correction to the original design, and the one most likely to earn credit.** The first class diagram put `aiScore` on `CandidateProfile` — one score per person. But a match score is meaningless without a job: the same backend engineer might score 95 against a Node.js role and 30 against a data science role. A score is a property of the *pairing*, so it needs its own entity keyed on both. Volunteering this — "here is a mistake I found in my own first draft and why it was wrong" — demonstrates exactly the review instinct interviewers are testing for.

3.4 Full-Time Hiring Pipeline

hiring_stages

The recruiter-defined pipeline for a specific job — "Screening", "Technical Round", "HR Round", "Offer" — with an explicit ordering.

Why per-job rather than a global enum: a sales role and an engineering role have genuinely different processes. Hard-coding stages as an enum would force every job into one shape and require a schema migration to add a stage. This table is the direct implementation of the "stage configuration" node in the mind map.

Design note. `UNIQUE(job_id, sequence)` prevents two stages claiming the same position. `is_terminal` marks end states so the UI knows which stages stop the pipeline.

applications

A candidate's formal application to a full-time role, and their current position in the pipeline.

Column	Notes
`current_stage_id` FK	Denormalized pointer to the latest stage, so the pipeline board renders without aggregating history on every load.
`source`	`DIRECT | INVITED` — lets you measure whether outbound invitations actually convert.
`snapshot_ai_score`	Point-in-time score at application. See §2.2.
`resume_id` FK	Pins the exact resume version submitted.

Likely question: "*You store current_stage_id but also a stage_movements history — isn't that redundant?*" Yes, and deliberately. The current stage is derivable from the latest movement, but the pipeline board reads it for every application on every page load. Deriving it means a correlated subquery or window function per row. The pointer is a cache; the history is the source of truth, and they are written in the same transaction.

stage_movements

An append-only audit log of every pipeline transition: from which stage, to which, by whom, when, and with what note.

Why it exists. It implements the mind-map requirements "candidate movement in the different stage" and "See all stage movement for that JD" from both sides. It also answers real analytics questions — average time in each stage, where candidates drop out — that are impossible if you only store current state. And because hiring decisions can be legally challenged, an immutable record of who moved whom and when is genuine risk mitigation.

Design note. `from_stage_id` is nullable: the first movement comes from nowhere.

invitations

Outbound recruiter-to-candidate outreach for a specific job — the reverse direction of an application.

Why not just create an application with `source = 'INVITED'`? Because consent runs the other way. An application is something a candidate authored; an invitation is something they have not yet accepted. Collapsing them would put candidates in pipelines they never agreed to join. Separating them also gives invitations their own lifecycle — `expires_at`, `declined` — that has no meaning for an application.

Design note. Shared across both flows: "send them an invite for the JD" appears for both full-time candidates and freelancers in the mind map, and the data is identical, so one table serves both.

offers

The formal offer extended after a successful pipeline, and the candidate's response.

Design note. One-to-one with `applications` (`UNIQUE(application_id)`) — you cannot make two competing offers on one application. `expires_at` matters commercially: offers lapse, and modelling that in the database rather than in a cron job's imagination keeps the state honest. This table backs the "See selected offers" screen on the candidate dashboard.

interviews OPTIONAL

Scheduled meetings within a pipeline stage, with structured feedback.

Column	Notes
`hiring_stage_id` FK	Ties the interview to its round. Without it you cannot tell a technical screen from a final round.
`round_no`	`UNIQUE(application_id, round_no)`.
`recommendation`	`PROCEED | HOLD | REJECT` — structured, so it is aggregatable. Free-text feedback alone cannot be reported on.

Design note. A CHECK enforces that an `ONLINE` interview has a meeting link, and that a `COMPLETED` one has a recommendation — the database refusing to record a finished interview with no outcome.

3.5 Freelance Engagement

proposals

A freelancer's competing bid on a project: their price, their timeline, and their pitch.

Why separate from `applications`? A proposal is a *negotiation*, an application is a *candidacy*. A proposal carries a bid amount and a duration that the client can accept or reject — commercial terms that have no analogue in full-time hiring, where compensation is set by the employer. Their lifecycles differ too: proposals are shortlisted then accepted; applications move through arbitrary configurable stages.

contracts

The binding agreement created when a client accepts a proposal. The anchor for all post-acceptance activity.

Column	Notes
`proposal_id` UNIQUE FK	One contract per accepted proposal.
`agreed_amount`	May differ from the original bid after negotiation — which is exactly why it cannot live on the proposal.
`platform_fee_percent`	Frozen at contract creation. If the platform later changes its commission, existing contracts honour the rate agreed at signing.
`status`	Includes `PENDING_FUNDING` — work should not begin before escrow is funded.

> **The most important structural fix in the entire schema.** The original class diagram attached milestones directly to `Proposal`. But a proposal is only an offer to work — most are rejected. Hanging milestones, escrow, chat, delivery, reviews and disputes off a rejected bid is structurally wrong, and it leaves nowhere to record the agreed amount, the frozen fee percentage, or the start and end dates. Introducing `contracts` as the boundary between *bidding* and *working* is the change to lead with when asked "what did you improve in your design?"

milestones

A contract split into independently funded, delivered and paid units of work.

Why milestones exist at all: they solve the fundamental trust problem of freelancing. The freelancer will not start unpaid; the client will not pay upfront. Milestones shrink the risk on both sides to one increment, and escrow removes it almost entirely. This is the schema encoding a business model, not just storing data — a good thing to say out loud.

Design note. The status enum is the state machine: `PENDING → FUNDED → SUBMITTED → APPROVED → PAID`, with branches to `REVISION_REQUESTED` and `DISPUTED`. Be ready to draw it.

milestone_submissions

Each delivery attempt against a milestone, versioned.

Why versioned rather than one submission per milestone: revision requests are normal, not exceptional. If you overwrite the submission each time, you destroy the evidence an admin needs to arbitrate a dispute — "the client claims nothing was delivered; here are three submissions with timestamps." The version history *is* the dispute evidence.

attachments

Files uploaded anywhere in the system — chat, submissions, dispute evidence, portfolios.

Design note: this table is polymorphic. `owner_type` plus `owner_id` identify the parent, which means the database cannot enforce a foreign key on it. This is a real trade-off and you should present it as one: one upload pipeline, one storage quota, one virus-scan hook and one deletion policy, purchased at the cost of referential integrity. The alternative — `message_attachments` and `submission_attachments` as separate tables — gives you real foreign keys but duplicates all of that logic. For a system where files are managed centrally, the polymorphic form wins; if integrity mattered more, I would split it.

3.6 Money — Escrow, Payments and the Ledger

escrows

Funds held by the platform on behalf of a milestone: deposited by the client, released to the freelancer on approval, refunded on cancellation.

Why one-to-one with a milestone: each milestone is independently funded, so the escrow lifecycle matches the milestone lifecycle exactly. A single escrow per contract would defeat the purpose — the freelancer would still be waiting on the whole project's funds.

Design note. `FROZEN` exists specifically for disputes: the moment one is raised, the escrow stops being releasable by either party until an admin resolves it. The status enum is doing real safety work here, not just describing state.

payments

Every inbound money movement from a user to the platform, across both flows.

Column	Notes
`purpose`	`CANDIDATE_PROFILE_FEE | JOB_POSTING_FEE | ESCROW_FUNDING` — one table covers all three fee types from the mind map.
`gateway_ref` UNIQUE	The provider's transaction ID. Unique, so a duplicated webhook cannot be processed twice — this is your idempotency key.
`status`	A payment is `INITIATED` before the gateway confirms. Never assume success at creation.

Likely question: "*How do you prevent double-charging if the payment webhook fires twice?*" The unique constraint on `gateway_ref` makes the second insert fail at the database level, regardless of what the application code does. Enforcing idempotency in the schema rather than in a service is the answer they are hoping for.

platform_fees

The platform's revenue lines, itemised by type: registration fees, posting fees, and freelance commission.

Why separate from `payments`: a payment is what the user paid; a fee is what the platform kept. On escrow funding the client pays 100 but the platform's revenue is only the commission on release. Conflating the two makes revenue reporting wrong. This table is what backs the "Platform Financials" admin view in the mind map.

wallets, transactions, payouts

The freelancer's earnings ledger and withdrawal mechanism.

Table	Role
`wallets`	Current balance, split into `available` and `pending`. One per user.
`transactions`	Immutable double-entry-style ledger. Every credit and debit, with `balance_after`.
`payouts`	Withdrawal requests to a bank account or UPI, with their own status lifecycle.

> **The concurrency answer — memorise this one.** A `CHECK (balance >= 0)` does *not* prevent double-spending. Under `READ COMMITTED`, two concurrent payout requests can both read a balance of 500, both pass the check, and both commit — leaving the wallet at −500 with the constraint never violated at any single moment either transaction observed. The fix is pessimistic locking: `SELECT ... FROM wallets WHERE id = $1 FOR UPDATE` at the start of the transaction, which serialises the two requests. The CHECK remains as a last-resort backstop, not as the primary control.

Why both a wallet and a ledger: the ledger is the source of truth and is append-only; the wallet is a materialized balance that gives you a row to lock and avoids summing thousands of rows on every read. A reconciliation job compares `SUM(credits) − SUM(debits)` against the stored balance and alerts on drift.

3.7 Communication

chat_rooms & chat_participants

A conversation thread attached to either a contract or an application, with an explicit participant list.

Design note. The original design linked chat rooms directly to both a candidate and a recruiter with two one-to-many edges — which is really an unresolved many-to-many, and offers no place to store per-user read state. Anchoring the room to a *contract* or *application* instead is better on three counts: it defines exactly who may access it, it gives the dispute-resolution admin the correct thread automatically ("the Admin needs a UI to review the chat" in the mind map), and it prevents chat before any working relationship exists — which is the standard anti-circumvention control on marketplaces, since otherwise both parties simply exchange contact details and transact off-platform.

The XOR constraint. A room must have exactly one parent, never both and never neither: `CHECK (num_nonnulls(contract_id, application_id) = 1)`.

messages

Individual chat messages, with read state and soft deletion.

Design note. `deleted_at` rather than a hard delete, because messages are dispute evidence. A freelancer should not be able to erase the conversation where they agreed to a scope change.

notifications

The user-facing event feed — invitations, stage changes, milestone approvals, payments.

Design note. `action_url` makes each notification clickable rather than merely informative. The partial index on unread rows keeps the badge count cheap, which matters because it is queried on literally every page load.

3.8 Trust, Disputes & Administration

reviews

Bidirectional ratings exchanged after a contract completes.

Design note. The original had two ambiguous one-to-many edges from candidate and recruiter, which cannot express *who reviewed whom*. Explicit `reviewer_id` and `reviewee_id` plus a `contract_id` fix that. `UNIQUE(contract_id, reviewer_id)` allows exactly one review per party per contract, and `CHECK (reviewer_id <> reviewee_id)` blocks self-review.

A product point worth raising unprompted: `is_published` enables double-blind release — neither review becomes visible until both are submitted or a deadline passes. Without it, whoever reviews second can retaliate against a bad rating, and the whole reputation signal degrades. Noticing that the schema can encode incentive design tends to land well.

disputes

A contested milestone escalated to an admin for arbitration.

Column	Notes
`milestone_id` FK	Attached to the milestone, not the contract — the mind map specifies disputes arise on a milestone, and only that milestone's escrow should freeze.
`assigned_admin_id`	Nullable until triaged.
`amount_to_freelancer` `amount_to_client`	The split decision. Real disputes rarely resolve fully in one party's favour — partial work deserves partial payment.

Design note. A CHECK enforces that a `RESOLVED` dispute must carry both amounts, so an admin cannot close a case without recording where the money went.

moderation_actions

The record of admin enforcement — warnings, suspensions, bans and their reversals.

Why an action log rather than a flag on `users`: both exist and serve different purposes. `users.account_status` is the current state, read on every login; `moderation_actions` is the history — who banned this person, when, why, and whether it was later revoked. A ban with no recorded reason is indefensible if the user appeals. `expires_at` supports temporary suspensions that lift automatically.

platform_settings

Global configuration as key–value pairs: fee percentages, file size caps, feature flags.

Why it beats a config file: changing the platform commission should not require a deployment. This is the "Some admin configuration" node in the mind map, and `updated_by` plus `updated_at` mean a surprise fee change is traceable to a person.

audit_logs

A generic, append-only record of significant mutations across the system.

Design note. Polymorphic like `attachments` (`entity_type` + `entity_id`), storing a `jsonb` diff of what changed. For a platform handling money and hiring decisions, being able to answer "who changed this contract's amount and when" is not optional. In production this table grows fastest of all — partition it by month and archive cold partitions to object storage.

4. Relationships & Cardinality Explained

4.1 Reading crow's-foot notation

Interviewers frequently ask you to read your own diagram aloud. The notation is directional: the symbol nearest each entity describes how many of *that* entity participate.

Symbol	Means	Example from this schema
`||--||`	Exactly one to exactly one	`users` to `wallets` — every user has precisely one wallet
`||--o|`	One to zero-or-one	`users` to `candidate_profiles` — a recruiter has none
`||--o{`	One to zero-or-many	`jobs` to `applications` — a new posting has none yet
`||--|{`	One to one-or-many	`contracts` to `milestones` — a contract with no milestones is meaningless
`}o--o{`	Many to many	Never drawn directly; always resolved through a junction table

> **The distinction that separates a careful diagram from a careless one** is `o|` versus `||`, and `o{` versus `|{`. Optionality is not decoration — it determines whether the foreign key column is `NULL` able, and therefore whether your application must handle a missing row. Saying "milestones is one-or-*many* because a contract with zero milestones has nothing to pay against, but applications is zero-or-many because a job starts with none" shows you thought about each edge rather than drawing them all identically.

4.2 The three many-to-many relationships

Relational databases cannot represent many-to-many directly; every one must be resolved through a junction table. This schema has three, and each junction carries its own attributes.

Logical relationship	Junction table	Attributes on the relationship itself
Candidates ↔ Skills	`candidate_skills`	proficiency, years of experience
Jobs ↔ Skills	`job_skills`	is_mandatory, weight
Projects ↔ Skills	`portfolio_project_skills`	none — a pure join

`chat_participants` is a fourth, resolving users to chat rooms and carrying `last_read_at` — per-user read state that has nowhere else to live.

4.3 The self-referencing relationships

Several relationships point back at `users` more than once, which is worth being able to explain because it looks like an error to an untrained eye.

- `reviews` references `users` twice — as `reviewer_id` and `reviewee_id`. The same table, two different roles.
- `contracts` references profiles twice — `client_id` to a recruiter and `freelancer_id` to a candidate.
- `stage_movements` references `hiring_stages` twice — as `from_stage_id` and `to_stage_id`.

When a table has two foreign keys to the same target, the *column name* carries the semantics, and each needs its own explicitly named constraint so error messages stay legible.

4.4 The dependency chain, in order

This is the order in which rows must be created, and therefore the order your migrations and your seed data must follow.

```
users
  └── candidate_profiles ── resumes
  │     └── freelance_portfolios ── portfolio_projects
  └── recruiter_profiles ── companies
        └── jobs ── hiring_stages, ai_weightages, job_skills
              │
              ├── FULL-TIME ─── applications ── stage_movements
              │                       ├── interviews
              │                       └── offers
              │
              └── FREELANCE ─── proposals ── contracts ── milestones
                                                  │            ├── milestone_submissions
                                                  │            ├── escrows ── payments
                                                  │            └── disputes
                                                  ├── chat_rooms ── messages
                                                  └── reviews
```

5. Eight Design Decisions You Must Be Able to Defend

Interviewers rarely ask "list your tables". They ask "why did you do it *that* way". These eight are the decisions in this schema most likely to be challenged, each with the reasoning and the alternative you rejected.

Decision 1 — Introducing `contracts` between proposal and milestone

What I did: milestones, escrow, chat, reviews and disputes all hang off a `contract`, which is created only when a proposal is accepted.

Why: a proposal is a *bid*, and most bids lose. Attaching the entire working relationship to a rejected bid is structurally wrong. More concretely, the agreed amount can differ from the bid after negotiation, and the platform's fee percentage must be frozen at signing so later pricing changes do not retroactively alter existing deals. Neither value has anywhere to live on a proposal.

Alternative rejected: promoting the accepted proposal in place, by adding status and amount columns to `proposals`. That conflates two lifecycles in one table and leaves every downstream foreign key pointing at a row that might be a rejected bid.

Decision 2 — Match score as its own entity, not a candidate attribute

What I did: `match_scores(job_id, candidate_id, score, breakdown)` rather than `candidate_profiles.ai_score`.

Why: compatibility is a property of a *pair*. A single score per candidate is not merely imprecise — it is meaningless, because the same person is an excellent match for one role and a poor one for another. The mind map's "see the AI matches of those candidates" is explicitly per-JD, so the original model could not have implemented the stated requirement.

Alternative rejected: computing scores on demand and never storing them. That makes ranking fifty candidates require fifty computations per page load, and destroys the audit trail you need when a rejection is challenged.

Decision 3 — One `jobs` table with a type discriminator

What I did: full-time and freelance postings share one table, separated by `job_type` and policed by a CHECK constraint.

Why: they share most columns, and crucially they share six relationships — skills, AI weightage, match scores, invitations, applications-or-proposals, and company. Splitting the table would double every one of those, or force a polymorphic reference that no foreign key can protect.

Cost I accept: some columns are null for some rows. The CHECK constraint contains the damage by making illegal combinations unstorable.

Say this if pushed: "This is class-table versus single-table inheritance. I chose single-table because the shared relationships outnumber the divergent columns. If the two job types diverged further — say freelance gained ten fields full-time never uses — I would revisit it."

Decision 4 — Escrow per milestone, not per contract

What I did: one escrow row per milestone, one-to-one.

Why: it is what makes incremental trust work. The client funds only the next increment, so their exposure is one milestone rather than the whole project; the freelancer sees funds already secured before starting. A single contract-level escrow would leave the freelancer waiting until the very end, which is the problem escrow exists to solve.

Decision 5 — A ledger plus a materialized balance

What I did: `transactions` is append-only and authoritative; `wallets.available_balance` is a maintained cache.

Why: the ledger gives correctness and auditability; the balance gives a single row to lock, which is what actually prevents double-spend. Deriving the balance on every read is correct but offers no lock target and gets slower with every transaction.

The follow-up you should pre-empt: they will drift eventually, through bugs or partial failures. A scheduled reconciliation job compares the two and alerts on mismatch. Saying "they can drift, and here is how I detect it" is far stronger than claiming they cannot.

Decision 6 — Chat anchored to a contract or application

What I did: a chat room belongs to exactly one contract or one application, enforced by an XOR check, with an explicit participant list.

Why: it defines access control structurally rather than through application logic, it hands a dispute admin the correct thread automatically, and it prevents contact before a working relationship exists — the standard control against parties taking the deal off-platform.

Decision 7 — Soft deletes on anything with legal or financial weight

What I did: `deleted_at` on users and messages rather than `DELETE`.

Why: deleting a user would cascade away contracts, payments and reviews that tax and consumer-protection rules require you to retain, and would silently rewrite the reputation of everyone who worked with them. Deleting messages would let a party destroy dispute evidence.

The nuance to acknowledge: this sits in tension with GDPR's right to erasure. The reconciliation is anonymisation rather than deletion — scrub the personal fields, retain the transactional skeleton. Naming that tension unprompted is a strong signal.

Decision 8 — Partial unique indexes instead of blanket UNIQUE

What I did: uniqueness on `(job_id, candidate_id)` applies only to live rows, via `WHERE status NOT IN ('WITHDRAWN','REJECTED')`.

Why: a blanket constraint means a candidate who withdraws can never reapply, even when the role is reposted six months later. The business rule is "no two *active* applications", and the constraint should say exactly that and nothing more.

Portability caveat worth mentioning: partial indexes are a Postgres feature. MySQL 8 requires emulating them with a generated column that evaluates to `NULL` for exempt rows, since `NULL`s escape unique enforcement.

6. Constraints — What the Database Enforces

6.1 Why constraints belong in the database

The standard objection is "we validate that in the application". The answer is that your application is not the only writer. Admin scripts, data migrations, background jobs, a support engineer in `psql`, a second service written by another team — all bypass your validation layer. A constraint is enforced on every path, forever, including paths that did not exist when you wrote it. Application validation gives better error messages; database constraints give guarantees. You want both, and they are not substitutes.

6.2 The four kinds used here

Kind	Guarantees	Example in this schema
PRIMARY KEY	Unique and not null — identifies the row	`id UUID PRIMARY KEY` on every table
FOREIGN KEY	Referential integrity — no orphans	`milestones.contract_id → contracts.id`
UNIQUE	No duplicate values or combinations	`UNIQUE(contract_id, reviewer_id)` on reviews
CHECK	A value satisfies a predicate	`CHECK (rating BETWEEN 1 AND 5)`

6.3 The constraints that encode business rules

These are the ones worth quoting in an interview, because each translates a product rule directly into something the database refuses to violate.

Constraint	Business rule it encodes
Partial unique on `applications`	One live application per candidate per job; reapplying after withdrawal is allowed
`UNIQUE(contract_id, reviewer_id)`	Each party reviews a contract once
`CHECK (reviewer_id <> reviewee_id)`	Nobody reviews themselves
`CHECK (weights sum to 100)`	AI scoring is always a complete distribution
`CHECK (num_nonnulls(contract_id, application_id) = 1)`	A chat room has exactly one parent
`CHECK` on job compensation shape	Full-time carries a salary band; freelance carries a budget; never both
`CHECK (salary_min <= salary_max)`	A salary range points the right way
`CHECK` on resolved disputes	An admin cannot close a case without recording the money split
`UNIQUE(milestone_id, version_no)`	Submission versions are sequential and gapless
`UNIQUE` on `gateway_ref`	A duplicated payment webhook cannot charge twice

6.4 Applying constraints to a table that already has data

A practical point that distinguishes someone who has actually shipped a migration.

```
-- 1. Find the rows that would violate the constraint, BEFORE adding it
SELECT job_id, candidate_id, COUNT(*)
FROM   applications
WHERE  status NOT IN ('WITHDRAWN','REJECTED')
GROUP  BY job_id, candidate_id
HAVING COUNT(*) > 1;

-- 2. Build the index without locking writes (cannot run inside a transaction)
CREATE UNIQUE INDEX CONCURRENTLY uq_applications_active
    ON applications (job_id, candidate_id)
    WHERE status NOT IN ('WITHDRAWN','REJECTED');

-- 3. For CHECK constraints on large tables, add unvalidated first,
--    then validate with a weaker lock that does not block writes
ALTER TABLE jobs ADD CONSTRAINT ck_job_compensation_shape
    CHECK (...) NOT VALID;
ALTER TABLE jobs VALIDATE CONSTRAINT ck_job_compensation_shape;
```

7. Indexing & Query Performance

7.1 What an index actually is

A B-tree index is a sorted structure over one or more columns that lets the database find matching rows without scanning the whole table — the difference between reading every page of a book and using its index. The cost is that every `INSERT`, `UPDATE` and `DELETE` must also update the index, and the index consumes disk. Indexing is therefore a deliberate trade of write speed and storage for read speed, applied where reads dominate.

7.2 What is indexed here, and why

Index	The query it exists for
`jobs(status, job_type, created_at DESC)`	The main job browse page — filter to open jobs of one type, newest first. Column order matters: equality filters first, the sort column last.
`applications(job_id, status)`	Rendering a recruiter's pipeline board for one job
`applications(candidate_id, applied_at DESC)`	The candidate's "my applications" list
`match_scores(job_id, score DESC) WHERE is_current`	The AI-ranked shortlist. Partial, so it indexes only live scores and stays small.
`messages(room_id, sent_at DESC)`	Loading the most recent page of a conversation
`notifications(user_id) WHERE NOT is_read`	The unread badge, queried on every page load. Partial index — read notifications, the vast majority, are never in it.
`transactions(wallet_id, created_at DESC)`	Earnings history

> **The composite index rule, stated properly.** A composite index on `(a, b, c)` can serve queries filtering on `a`, on `a, b`, or on `a, b, c` — but not on `b` alone. It is a sorted list ordered by `a` first, so entering it without a value for `a` is impossible. This is the leftmost-prefix rule, and it explains why column order in a composite index is a decision rather than an accident.

7.3 Indexes you should *not* add

- **On low-cardinality columns alone.** An index on a boolean is usually useless — if half the rows match, a sequential scan is cheaper than the index lookup plus row fetch. Use a partial index on the rare value instead.
- **Duplicating a leftmost prefix.** With `(job_id, status)` already present, a separate index on `(job_id)` is dead weight that still costs write time.
- **Speculatively.** Add indexes in response to a measured slow query, not in anticipation. Every unused index is pure write overhead.

8. End-to-End Walkthroughs with SQL

Being able to trace a complete user journey through the tables — naming each write in order — is the clearest possible demonstration that you designed the schema rather than copied it.

8.1 Full-time hire, from posting to offer

Step	What happens	Rows written
1	Recruiter creates a posting	`jobs` (status `DRAFT`), `job_skills`, `hiring_stages` ×4, `ai_weightages`
2	Pays the posting fee	`payments`, `platform_fees`; `jobs.status → OPEN`
3	System scores candidates	`match_scores` per candidate, `is_current = true`
4	Recruiter invites a top match	`invitations`, `notifications`
5	Candidate accepts and applies	`applications` (source `INVITED`, resume pinned), `stage_movements` (null → Screening)
6	Moved to Technical Round	`stage_movements`, `applications.current_stage_id`, `notifications`
7	Interview scheduled and held	`interviews`, then feedback and `recommendation`
8	Offer extended and accepted	`offers`, `applications.status → HIRED`, `jobs.status → CLOSED`

The AI-ranked shortlist query

```
SELECT u.full_name,
       cp.headline,
       ms.score,
       ms.breakdown,
       a.status,
       hs.name AS current_stage
FROM   match_scores      ms
JOIN   candidate_profiles cp ON cp.id = ms.candidate_id
JOIN   users              u  ON u.id  = cp.user_id
LEFT   JOIN applications  a  ON a.candidate_id = cp.id
                            AND a.job_id       = ms.job_id
LEFT   JOIN hiring_stages hs ON hs.id = a.current_stage_id
WHERE  ms.job_id     = $1
  AND  ms.is_current
  AND  u.account_status = 'ACTIVE'
ORDER  BY ms.score DESC
LIMIT  50;
```

The `LEFT JOIN` is deliberate: the shortlist must include strong matches who have *not* applied, since those are exactly the people the recruiter wants to invite. An inner join would hide them — a subtle bug worth pointing out if asked to critique the query.

8.2 Freelance project, from proposal to payout

Step	What happens	Rows written
1	Client posts a project	`jobs` (`FREELANCE`, budget, `budget_type`)
2	Freelancers bid	`proposals` — one live row per freelancer
3	Client accepts one	`proposals.status → ACCEPTED`, `contracts`, `milestones` ×3, `chat_rooms` + `chat_participants`
4	Client funds milestone 1	`payments` (`ESCROW_FUNDING`), `escrows` (`HELD`), `milestones.status → FUNDED`
5	Work delivered	`milestone_submissions` v1, `attachments`, `messages`
6	Revision requested, then v2	`milestone_submissions` v2 — v1 retained as evidence
7	Approved	`escrows → RELEASED`, `platform_fees` (commission), `transactions` CREDIT, `wallets.available_balance` increased
8	Contract completes	`reviews` ×2, `contracts.status → COMPLETED`, portfolio aggregates refreshed
9	Freelancer withdraws	`payouts`, `transactions` DEBIT, balance decreased

The milestone release transaction — write this one out if asked

```
BEGIN;

-- Lock the wallet row first; this is what serialises concurrent writers
SELECT available_balance
FROM   wallets
WHERE  user_id = $freelancer_user_id
FOR    UPDATE;

UPDATE milestones SET status = 'APPROVED', approved_at = now()
WHERE  id = $milestone_id AND status = 'SUBMITTED';

UPDATE escrows SET status = 'RELEASED', released_at = now()
WHERE  milestone_id = $milestone_id AND status = 'HELD';

INSERT INTO platform_fees (contract_id, fee_type, amount, percentage, charged_at)
VALUES ($contract_id, 'FREELANCE_COMMISSION',
        $amount * $fee_pct / 100, $fee_pct, now());

UPDATE wallets
SET    available_balance = available_balance + ($amount * (100 - $fee_pct) / 100)
WHERE  user_id = $freelancer_user_id;

INSERT INTO transactions (wallet_id, direction, amount, balance_after,
                          reference_type, reference_id, description)
VALUES ($wallet_id, 'CREDIT', $net, $new_balance,
        'ESCROW', $escrow_id, 'Milestone released');

COMMIT;
```

Two things to say about this block. First, every statement is in one transaction, so a crash between the escrow release and the wallet credit rolls everything back — money is never created or destroyed by a partial failure. This is atomicity, demonstrated rather than recited. Second, the `UPDATE ... WHERE status = 'SUBMITTED'` is a guard: if another request already approved this milestone, the update matches zero rows and the application detects the no-op instead of paying twice.

8.3 A dispute, and how the schema supports arbitration

When a client refuses to approve a milestone, the freelancer raises a dispute. The escrow moves to `FROZEN` — neither party can move the money. The admin assigned to the case has everything they need already linked: the chat room via the contract, every submission version with timestamps, the milestone description that defines the agreed scope, and any evidence attachments. Resolution writes `amount_to_freelancer` and `amount_to_client`, and a CHECK constraint refuses to let the case close as `RESOLVED` without both.

This is the strongest illustration of the guide's recurring theme: *good schema design is not about storing data, it is about making the required workflows possible and the invalid states unrepresentable.* If chat had been attached to users rather than contracts, or submissions overwritten rather than versioned, the admin would be arbitrating on missing evidence.

9. Interview Question Bank

Forty questions, grouped by what they are actually testing. The answers are written the way you would say them aloud — direct first sentence, reasoning second.

9.1 Opening and scope questions

1. Walk me through your project.

> It is a recruitment platform with two flows sharing one schema. In the full-time flow a recruiter posts a role, configures a pipeline and AI scoring weights, and moves candidates through stages to an offer. In the freelance flow a client posts a project, freelancers bid, and the accepted bid becomes a contract split into milestones with escrowed payments released on approval. Forty-one tables in Postgres. The parts they share — users, jobs, skills, AI matching, chat — are single tables; the parts that diverge get their own.

2. Why did you build both flows instead of one?

> Because they are genuinely different products for the same audience, and the shared surface is large — identity, company data, skills taxonomy, the matching engine and messaging are all identical. Building them separately would have meant duplicating all of that. The interesting engineering was deciding exactly where the shared path ends, and the answer is: at the point of commitment. Applications and proposals diverge, and everything downstream does too.

3. How long did this take, and what was the hardest part?

> Be honest about the timeline. For the hard part, the strongest answer is the money flow — specifically realising that a `CHECK (balance >= 0)` does not prevent double-spending under concurrent requests, and that the actual fix is row-level locking. That is a real lesson rather than a rehearsed one.

9.2 Schema design fundamentals

4. What normal form is your database in?

> Third normal form, with four documented denormalizations. I can give an example of each normal form from my own tables: skills extracted from a comma-separated string for 1NF; proficiency depending on both candidate and skill in the junction table for 2NF; company data extracted out of the recruiter profile to remove a transitive dependency for 3NF.

5. Where did you denormalize, and why is that acceptable?

> Four places, each for a different reason: average rating on portfolios for search read performance; the AI score snapshot on applications for point-in-time legal accuracy; balance-after on transactions for audit; and the wallet balance for concurrency control. Denormalization is acceptable when it is deliberate, documented, and has a defined path back to the source of truth. It is a problem when it is accidental.

6. Why UUIDs instead of auto-increment integers?

> These IDs appear in URLs and are shared between two parties, so sequential integers would let anyone enumerate every job and application, and would leak business volume. The cost is 16 bytes per key and scattered index writes. If write throughput became a problem I would move to UUIDv7, which is time-ordered and restores insert locality while staying non-guessable.

7. Why is skills a separate table rather than a text column?

> 1NF. A comma-separated string cannot be indexed, cannot be filtered reliably — searching "Java" would match "JavaScript" — and gives nowhere to store proficiency. Extracting it also enables a controlled vocabulary, which stops "React", "ReactJS" and "react" fragmenting into three separate facets that each under-report.

8. What is a junction table and where do you use one?

> It resolves a many-to-many relationship, which relational databases cannot express directly. I have three, and each carries attributes belonging to the relationship itself rather than to either side — proficiency in `candidate_skills` is not a property of the candidate or of the skill, but of the pairing. That makes them associative entities, not just join tables.

9. Explain the difference between one-to-one and one-to-zero-or-one in your schema.

> `users` to `wallets` is one-to-one — every user gets a wallet at registration. `users` to `candidate_profiles` is one-to-zero-or-one, because a recruiter never has one. The practical consequence is whether the application must handle a missing row, and whether the foreign key column is nullable.

10. Why separate candidate_profiles from users?

The three roles need very different attributes. One combined table would leave most columns null for most rows. Separating them also keeps the users row narrow, which matters because it is read on every authenticated request.

9.3 The design decisions most likely to be challenged

11. Why do milestones belong to a contract rather than a proposal?

A proposal is a bid, and most bids are rejected — you should not hang escrow, chat and payments off a rejected bid. Beyond structure, the contract holds two values that have nowhere else to live: the agreed amount, which can differ from the bid after negotiation, and the platform fee percentage frozen at signing so later pricing changes do not alter existing deals. This was actually a fix to my first draft.

12. Why is the AI score its own table?

Because a match score is a property of a candidate–job *pair*. My first version stored a single score per candidate, which is meaningless — the same engineer is a 95 for one role and a 30 for another. It also could not have implemented the requirement to see AI matches per job posting.

13. Why one jobs table rather than two?

They share roughly eighty percent of their columns and, more importantly, six relationships — skills, weightages, match scores, invitations, company and status. Splitting the table doubles all of those or forces a polymorphic reference no foreign key can protect. The price is a few nullable columns, which a CHECK constraint keeps honest. It is single-table inheritance, chosen knowingly.

14. Why is chat attached to a contract instead of to the two users?

Three reasons. Access control becomes structural rather than a rule in application code. The dispute admin gets the right thread automatically. And it prevents users exchanging contact details before any working relationship exists, which is the standard control against off-platform circumvention — the platform's revenue depends on it.

15. Your attachments table has no foreign key. Isn't that bad design?

It is a deliberate trade-off. It is polymorphic, so the database cannot enforce the reference. What I get is one upload pipeline, one quota, one virus-scan hook and one deletion policy across chat, submissions, disputes and portfolios. The alternative — separate tables per parent — gives real foreign keys but duplicates all of that. If integrity mattered more than centralisation I would split it, and I would enforce the reference with a trigger in the meantime.

16. You store current_stage_id and a full movement history. Isn't that redundant?

Yes, and deliberately. The pipeline board reads the current stage for every application on every load; deriving it would need a correlated subquery per row. The history is the source of truth and the pointer is a cache, and both are written in the same transaction so they cannot diverge.

17. Why keep every submission version instead of overwriting?

Revisions are normal in freelance work, and the version history is the evidence an admin needs to arbitrate a dispute. If the client claims nothing was delivered, three timestamped submissions settle it. Overwriting destroys exactly the record you need at exactly the moment you need it.

18. Why soft delete users instead of deleting them?

A hard delete would cascade away contracts, payments and reviews that must be retained for tax and consumer-protection purposes, and would silently rewrite the reputation of everyone who worked with that person. The tension with GDPR's right to erasure is real, and the reconciliation is anonymisation — scrub personal fields, keep the transactional skeleton.

9.4 Constraints, integrity and correctness

19. Why put constraints in the database if the application already validates?

Because the application is not the only writer. Migrations, admin scripts, background jobs and anyone with a database console all bypass it. A constraint holds on every path, including ones that did not exist when it was written. Application validation gives better error messages; constraints give guarantees. You want both.

20. How do you stop a candidate applying to the same job twice?

A partial unique index on `(job_id, candidate_id)` restricted to rows whose status is not withdrawn or rejected. A blanket unique constraint would permanently block someone who withdrew from ever reapplying, even to a role reposted a year later. The rule is "no two active applications", so the constraint should say exactly that.

21. How do you guarantee AI weights are valid?

A CHECK that the four weights sum to 100, plus range checks on each. Without it a recruiter could enter weights summing to 40 and every score would come out proportionally low with no error anywhere — a silent failure, which is the worst kind.

22. A chat room can point at a contract or an application. How do you stop both being set?

`CHECK (num_nonnulls(contract_id, application_id) = 1)` in Postgres. SQL has no XOR operator, so on MySQL it becomes `(contract_id IS NULL) <> (application_id IS NULL)`. I also added a check tying `room_type` to whichever parent is set.

23. How would you add a constraint to a table with millions of existing rows?

First query for violating rows and clean them. Then `CREATE UNIQUE INDEX CONCURRENTLY` for unique constraints, which does not block writes but cannot run inside a transaction. For CHECK constraints, add them `NOT VALID` so they apply to new rows immediately with only a brief lock, then `VALIDATE CONSTRAINT` separately under a weaker lock.

9.5 Concurrency, transactions and money

24. Two payout requests arrive at once for a wallet with 500. What happens?

Without protection, both read 500, both pass the non-negative check, and both commit — the wallet ends at minus 500 and the constraint was never violated at any instant either transaction observed. The fix is `SELECT ... FOR UPDATE` on the wallet row at the start of each transaction, which forces the second to wait for the first to commit and then re-read the true balance. The CHECK stays as a backstop, not as the control.

25. What is a transaction, and where does it matter most here?

A unit of work that either fully commits or fully rolls back. It matters most in the milestone release: escrow status, platform fee, wallet balance and ledger entry all change together. A crash halfway through without a transaction would release the escrow without crediting the freelancer — money vanishing from the system.

26. Explain ACID using your own schema.

Atomicity — the milestone release either completes entirely or not at all. **Consistency** — constraints mean no committed state has an escrow released without a matching ledger entry. **Isolation** — two recruiters moving the same candidate do not interleave into a corrupt state. **Durability** — once the payment commits, a power failure cannot undo it.

27. How do you make payment webhooks idempotent?

A unique constraint on `gateway_ref`. If the provider delivers the same webhook twice, the second insert fails at the database level regardless of what the handler code does. The handler catches the duplicate-key error and returns success, because the work was already done.

28. Why store balance_after on every transaction?

Auditability. An auditor can verify any single line without replaying the entire history, and a mismatch between consecutive rows immediately localises where corruption began. Standard practice in financial ledgers.

29. What happens if the wallet balance drifts from the ledger?

It will eventually, through bugs or partial failures. A reconciliation job periodically computes the sum of credits minus debits per wallet and compares it against the stored balance, alerting on any mismatch. The ledger is authoritative, so recovery means recomputing the cache. Claiming drift is impossible would be the wrong answer; detecting it is the right one.

9.6 Performance and scale

30. Which queries did you index for, and why those?

The ones on hot paths: job browse, the recruiter pipeline board, the AI shortlist, the message thread and the unread-notification badge. I did not index speculatively — every index costs write time, and an unused one is pure overhead.

31. Why is the job browse index ordered (status, job_type, created_at)?

Equality filters first, the sort column last. The index is sorted by status, then type, then date, so a query filtering on the first two and sorting by the third reads a contiguous range with no separate sort step. Reversing the order would make it useless for that query.

32. What is a partial index and where did you use one?

An index over only the rows matching a predicate. I used one for unread notifications — read notifications are the overwhelming majority and are never queried by the badge, so excluding them keeps the index small enough to stay cached. Also for current match scores and for the active-row uniqueness rules.

33. Your platform has a million candidates. Scoring every one against a new job is slow. What now?

Do not score everyone. Pre-filter on mandatory skills, location and availability using indexed columns, which typically cuts the candidate set by orders of magnitude, then score only the survivors. Run it asynchronously on a queue rather than during the request, and cache with `is_current` so it is computed once per candidate–job pair rather than per page view. At real scale the pre-filter itself moves to a dedicated search index.

34. Which table grows fastest and how do you handle it?

`audit_logs`, then `notifications` and `messages`. Partition by month, keep recent partitions hot and archive older ones to object storage. Partitioning also makes deletion cheap — dropping a partition is instant where deleting millions of rows is not.

35. How would you scale reads?

Read replicas for anything tolerant of slight staleness — job browse, search, analytics — while keeping money and pipeline writes on the primary. Cache the derived reputation aggregates. Beyond that the sharding key would be tricky, since jobs and candidates cross-reference; I would reach for replicas and caching long before sharding.

9.7 Security, privacy and edge cases

36. How are passwords stored?

Hashed with bcrypt or Argon2id, never encrypted and never plain. The column is `password_hash` so nobody misreads its purpose. Fast hashes like MD5 or bare SHA-256 are wrong here precisely because they are fast — slowness is the security property.

37. How do you stop one candidate reading another's application?

Authorization at the query layer: every access is scoped by the authenticated user's profile ID, and chat access derives from membership in `chat_participants`. The schema helps by making ownership explicit on every row and by using non-enumerable UUIDs, so a guessed URL is not a viable attack.

38. A user requests deletion of their data. What do you do?

Anonymise rather than delete. Contracts, payments and reviews must survive for financial and legal reasons and because they encode the counterparty's reputation. So personal fields are scrubbed, the account is marked deleted, and the transactional skeleton remains. This is a real tension between the right to erasure and record-keeping obligations, and anonymisation is the standard reconciliation.

39. What stops a recruiter and freelancer taking the deal off-platform?

Structurally, chat only exists once a contract or application exists, so there is no channel before commitment. Beyond the schema you would scan messages for contact details and keep escrow attractive enough that bypassing it means losing payment protection. This is the central commercial risk for any marketplace, and it is worth naming as a product problem rather than only a technical one.

40. What would you build next?

Saved searches with job alerts, since the matching data already exists and only needs a subscription table. Then analytics on the pipeline — time-in-stage and drop-off rates — which `stage_movements` already makes computable without any schema change. Designing so that the next feature needs no migration is worth pointing out as a deliberate outcome.

10. Known Trade-offs — Answering "What Would You Change?"

Almost every interview ends here, and it is the question candidates handle worst. Two failure modes: claiming the design is fine as-is, which reads as inability to self-review, or listing flaws with no reasoning, which reads as low confidence. The correct shape is: *here is the limitation, here is why I accepted it at the time, here is what would trigger changing it.*

Limitation	Why I accepted it	What would make me change it
Polymorphic `attachments` has no enforceable FK	One upload, quota, scanning and deletion pipeline instead of four	Any orphaned-file incident in production; I would split per parent or add a trigger
Admin permissions as JSON, not an RBAC model	A small fixed admin team; avoids three extra tables for little gain	A growing admin org, or needing to query "which admins can issue refunds"
Single currency assumed in most logic	Launch market is domestic; currency columns exist but no conversion layer	The first international client; needs an exchange-rate table and rate-at-transaction-time storage
No full-text search infrastructure	Postgres `tsvector` is adequate at current volume	Search latency past roughly 100ms, or a need for fuzzy matching and relevance tuning
Notifications stored but no delivery channel modelled	In-app feed was the MVP requirement	Adding email or push, which needs per-channel preferences and delivery status
Reviews are one-dimensional (a single rating)	Simple to build and to read	Users wanting to distinguish communication from quality; would become a sub-ratings table
No rate limiting or abuse tables	Out of scope for the data model	The first spam incident; proposal flooding is a known marketplace attack
Escrow assumes a single funding source	Matches the current payment provider integration	Split payments or partial refunds, which would need an escrow-to-payment many-to-many

> **The closing line that works:** "The design I would most want to revisit is the polymorphic attachments table — I traded referential integrity for a single file pipeline, and I think that was right at this scale, but it is the one place where the database cannot protect me and I am relying on application discipline instead."
>
> That answer identifies a specific weakness, explains the reasoning, states a position, and acknowledges what it depends on. That is what the question is testing.

11. Glossary

Term	Meaning in this project
Entity	A thing the system stores information about; becomes a table
Attribute	A property of an entity; becomes a column
Primary key	The column uniquely identifying each row — here always `id`
Foreign key	A column pointing at another table's primary key, enforcing that the target exists
Cardinality	How many rows on each side of a relationship participate — one-to-one, one-to-many, many-to-many
Junction / associative entity	A table resolving many-to-many, often carrying attributes of the relationship itself, e.g. `candidate_skills`
Normalization	Organising tables to eliminate redundancy and update anomalies
Denormalization	Deliberately reintroducing redundancy for performance or point-in-time accuracy
Transitive dependency	A non-key column depending on another non-key column — what 3NF removes
Constraint	A database-enforced rule: PRIMARY KEY, FOREIGN KEY, UNIQUE or CHECK
Partial index	An index covering only rows matching a predicate, keeping it small and cache-resident
Composite index	An index over several columns, usable only from its leftmost prefix
Transaction	A unit of work that commits entirely or rolls back entirely
Pessimistic locking	Locking a row with `SELECT ... FOR UPDATE` so concurrent writers serialise
Idempotency	The property that repeating an operation has no additional effect — here via unique `gateway_ref`
Soft delete	Marking a row deleted with `deleted_at` rather than removing it
Escrow	Funds held by the platform between client payment and freelancer release
Discriminator	A column indicating which subtype a row is, e.g. `jobs.job_type`
Single-table inheritance	Storing subtypes in one table with a discriminator and nullable columns
Polymorphic association	A reference identified by type plus ID, which no foreign key can enforce
Materialized balance	A stored aggregate kept in sync with an authoritative ledger
Point-in-time snapshot	A value frozen at a moment because later recomputation would be misleading

Final preparation advice. You will not be asked forty questions. You will be asked three or four, and the difference between a good and a poor answer is whether you give the *reason* unprompted. "Milestones belong to contracts" is a fact. "Milestones belong to contracts because a proposal is just a bid and most bids are rejected, and because the agreed amount can differ from the bid after negotiation" is an answer that ends the follow-up questions.

Practise saying the eight decisions in §5 out loud until each takes about thirty seconds. That is the highest-yield thing you can do with this document.